

Benchmarking Multi-Agent Machine Learning for Augmented Reality Systems

Klea Meta — Parthkumar Joshi
CSC 336
Professor Saptarashmi Bandyopadhyay

Abstract—Augmented Reality (AR) agents need to be fast and smart at the same time, but this is really hard to pull off. On one hand, they need to react instantly (in milliseconds) without using too much data. On the other hand, they need powerful AI models to understand complex surroundings. Running everything on the cloud allows us to use huge models, but it causes lag if the internet is slow. Running models directly on the device solves the lag, but mobile chips usually aren't strong enough for the best AI.

This paper tests a Teacher-Student Distillation method to fix this problem. We want to see if a huge "Teacher" model (running on a powerful computer) can teach a tiny "Student" model (running on a mobile device) to do the same job but much faster.

We tested this in three phases. Phase 1 used a simple grid game where the agent had to find a goal. The student learned to copy the teacher perfectly (100% success) but ran $78\times$ faster. Phase 2 used a vision-based navigation game. It was much harder, but the student actually performed slightly better than the teacher (13% vs 11%) while being way smaller. Phase 3 used real videos from the EGTEA Gaze+ dataset. Using a mix of learning methods, our small MobileNet student actually beat the large CLIP teacher (36.20% accuracy vs 22.27%) and ran $14\times$ faster.

I. INTRODUCTION

Imagine AR glasses that can give you directions or help you cook in real-time. For this to work, the AI inside them needs to be "always on." But building this is a huge technical challenge because mobile devices have strict limits.

State-of-the-art vision models, like Large Vision-Language Models (LVLMs) or Video Transformers, are very heavy. They require massive computational power that simply doesn't exist on a pair of glasses. This creates a dilemma with three main problems:

- **Latency (Lag):** If we send video to the cloud, it takes time to travel back and forth. In AR, even a slight delay can make the user feel sick or make the virtual objects look like they are sliding around.
- **Bandwidth:** Streaming high-quality video to a server constantly uses a massive amount of data. If the Wi-Fi drops, the agent stops working.
- **Battery Life:** Running heavy AI models on the glasses would drain the battery in minutes and likely cause the device to overheat, making it uncomfortable to wear.
- **Privacy:** Sending a constant video stream of your life to a cloud server is a major privacy risk. Processing data locally on the device is much safer.

In this project, we look at a **Teacher-Student Distillation** setup as a solution. Ideally, a big, powerful Teacher model

(simulating a cloud computer) generates data to train a small, efficient Student model that lives on the glasses.

Our main question is:

"Can we use a big cloud model to train a small on-device model that is fast enough for AR but still smart enough to work?"

We tested this in three distinct ways: simple movement (symbolic control), basic visual navigation (synthetic vision), and recognizing actions in real-world video (egocentric perception).

II. RELATED WORK

A. Knowledge Distillation

Knowledge Distillation is a technique introduced by Hinton et al. [1]. It's basically transferring "knowledge" from a big network (Teacher) to a smaller one (Student). Usually, when an AI is trained, it is told the "Hard Answer" (e.g., "This is a cat"). But a Teacher model can provide "Soft Answers" (e.g., "This is 90% a cat, 9% a dog, and 1% a car"). These soft probabilities contain extra information about how similar different objects look. We are trying to use this concept for Egocentric AR, where speed is the most important thing.

B. Egocentric Vision

Egocentric vision means looking at the world from a first-person view, like a GoPro or smart glasses. This is harder than normal computer vision because:

- The camera shakes a lot as the person moves.
- The user's hands often block the view of the object they are interacting with.
- Backgrounds are messy and cluttered.

Most top models here use heavy 3D processing that is too slow for mobile, so we want to compress that knowledge into simpler networks.

C. Mobile Architectures

We used **MobileNetV3** as our student. This is a chip design made specifically for phones. It uses "depthwise separable convolutions." In simple terms, instead of doing one big, heavy calculation on an image, it breaks the math down into smaller, separate channels. This drastically reduces the number of calculations needed, making it perfect for battery-powered devices.

III. METHODOLOGY

Our approach combines three different tasks under one main idea: making things faster for the edge (mobile devices).

A. The Teacher–Student Architecture

We have two main pathways in our experiment:

- 1) **The Cloud Pathway (Teacher):** A high-capacity model $\mathcal{T}$ (e.g., PPO-MLP or CLIP-ViT). This model assumes we have unlimited computer power, like a server with big GPUs. It learns the task perfectly first.
- 2) **The Edge Pathway (Student):** A mobile-optimized architecture $\mathcal{S}$ (e.g., Compressed MLP or MobileNetV3). This model is designed to run on constrained hardware like glasses.

Deployment Mode: When the user is actually using the AR glasses, only the Student is running. The heavy Teacher is turned off. This shifts the work from the cloud to the edge, cutting out the network lag entirely.

B. How We Train (Hybrid Loss)

To make sure the Student learns well, we use a **Hybrid Loss**. This means the Student is trying to minimize two errors at the same time:

- **Distillation Loss:** The difference between the Student’s guess and the Teacher’s guess. This helps the Student mimic the “style” of the Teacher.
- **Ground Truth Loss:** The difference between the Student’s guess and the actual correct answer (from the dataset).

This combination is crucial. If the Teacher makes a mistake, the “Ground Truth Loss” helps the Student correct it. If the dataset is confusing, the “Distillation Loss” helps the Student stay stable.

IV. PHASE 1: SYMBOLIC GRIDWORLD

A. Experimental Setup

Environment: A simple 5×5 grid. The state is a list of numbers: coordinates for the agent and coordinates for the goal ($x_{agent}, y_{agent}, x_{goal}, y_{goal}$).

Process: The agent sees its coordinates and decides to move Up, Down, Left, or Right. It receives a reward of +1 if it hits the goal. It loses a tiny bit of points (-0.01) for every step it takes, so it is motivated to find the shortest path possible.

We trained the Teacher using an algorithm called **PPO (Proximal Policy Optimization)**. This is a standard Reinforcement Learning method. Once the Teacher was perfect, we gathered 10,000 examples of it playing the game. We then trained the small Student network (a tiny MLP) to clone this behavior.

B. Results

We compared them on Success Rate (did they reach the goal?) and Latency (how fast did they think?).

Analysis: As you can see in Table I, the Student learned perfectly. The biggest difference was speed: the Student was **78×**

TABLE I

PHASE 1 RESULTS. THE STUDENT IS JUST AS GOOD BUT MUCH FASTER.

Metric	Teacher (PPO)	Student (Distilled)
Success Rate	100%	100%
Avg. Steps	6.2	6.2
Time per Step	110.0 ms	1.4 ms
Bandwidth	High	Zero (Local)

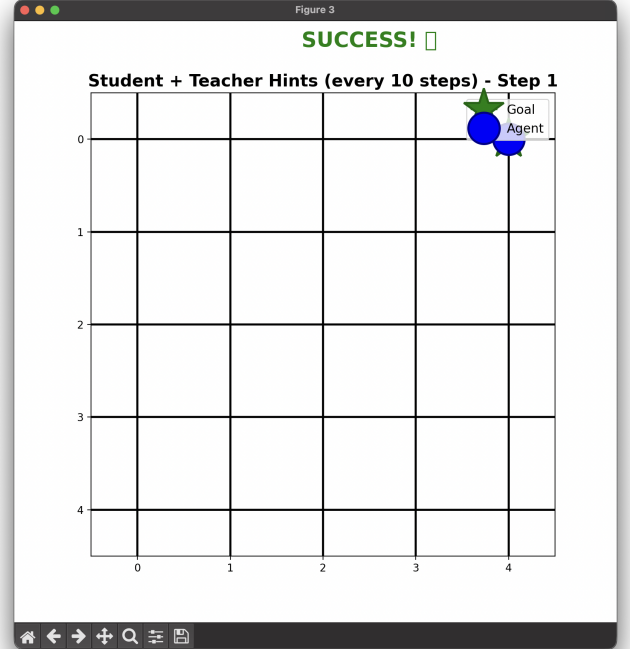


Fig. 1. Student/Teacher Gridworld Training

faster (1.4ms vs 110ms). This proved our main idea works: moving the “brain” of the agent from a Python simulation to a local, compiled model eliminates the major bottleneck of system overhead.

V. PHASE 2: VISUAL NAVIGATION

A. Why We Didn’t Use Realistic Sims

At first, we tried to use high-quality 3D simulators (like Habitat or AI2-THOR) to make it look like real AR. However, it didn’t work well. We ran into big problems:

- 1) **Sparse Rewards:** The agent only got points when it reached the goal. In a big 3D room, it basically wandered around aimlessly and never learned anything.
- 2) **Too Slow:** Rendering realistic 3D graphics took too long. We couldn’t run the millions of training steps needed for the AI to get smart.

Because of this, we switched to a simpler custom visual environment so we could focus on testing the distillation part without worrying about the graphics.

B. Synthetic Environment Setup

Environment: An 8×8 grid, but instead of coordinates, the agent sees a first-person pixel image (32×32 RGB).

Process: The model uses a Convolutional Neural Network (CNN) to "see" the image.

- **Teacher:** A Deep Q-Network (DQN) with a 3-layer CNN. It outputs "Q-values," which are estimates of how good a move is.
- **Student:** A compressed CNN ($16\times$ smaller). It tries to predict the same Q-values as the teacher.

C. Results

Performance: This task was hard. The Teacher only succeeded 11% of the time. Surprisingly, the Student got **13%**.

TABLE II
PHASE 2 RESULTS (VISUAL NAVIGATION).

Metric	Teacher (DQN)	Student (CNN)
Success Rate	11%	13%
Action Match	N/A	22.39%
Latency	106 ms	2.7 ms
Parameters	422,000	26,000

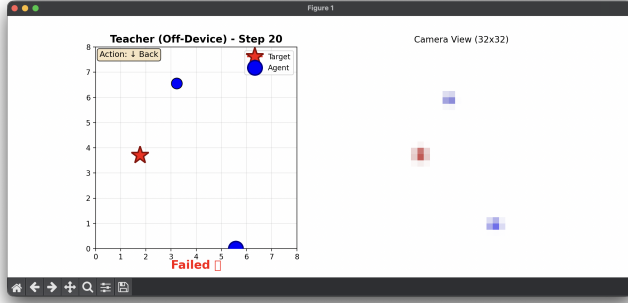


Fig. 2. AR Vision Training Process

What happened? We checked how often the Student picked the *exact same* move as the Teacher ("Action Match"). It was only about 22%. But since the Student had a similar success rate, it means the Student didn't memorize the Teacher's moves perfectly—it just learned how to solve the game on its own. This is called "Functional Equivalence."

VI. PHASE 3: REAL VIDEO RECOGNITION

A. Dataset and Task

We utilized the **EGTEA Gaze+** dataset. This dataset contains videos of people doing kitchen tasks (like cutting food, washing dishes, or opening fridges) from a first-person perspective.

Process: We focused on **Verb Classification** (identifying what action is happening, like "Cut" vs. "Wash"). Since video is heavy, we used a sampling technique: 1. We take a video

clip. 2. We pick 8 frames from that clip. 3. The model looks at these 8 frames to make a decision.

The Models:

- **Teacher:** We used a model called **CLIP (ViT-L/14)**. This is a massive "Vision Transformer" trained on the internet. It looks at the frames and creates a "video embedding" (a mathematical summary of the video).
- **Student:** We used **MobileNetV3-Large**. It tries to match the Teacher's summary AND the actual correct label using our Hybrid Loss.

B. Results: Student Beating the Teacher

This was the most interesting part of the project. As Table III shows, the Student actually did better than the Teacher.

TABLE III
PHASE 3 RESULTS. THE STUDENT WINS SIGNIFICANTLY.

Split	Teacher Accuracy	Student Accuracy	Difference
Split 1	22.20%	35.80%	+13.6%
Split 2	22.20%	35.40%	+13.2%
Split 3	22.40%	37.40%	+15.0%
Average	22.27%	36.20%	+13.93%

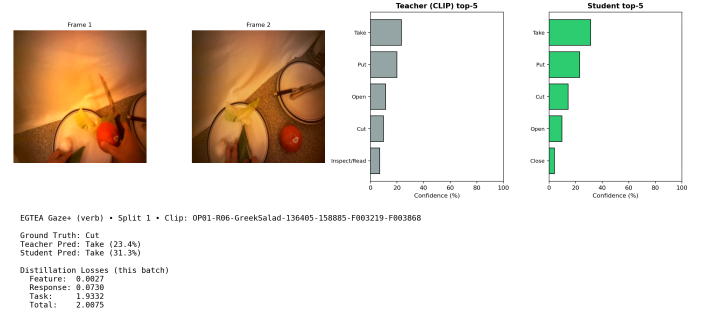


Fig. 3. EGTEA Verb Annotation Training

C. Why did the Student Win?

We believe this happened because of something called "Inductive Bias." The Teacher (ViT-L) is a Transformer. Transformers are great, but they need huge amounts of data to learn patterns. Our dataset was relatively small. The Student (MobileNet) is a CNN. CNNs are naturally good at looking at pictures and finding shapes/edges regardless of where they are in the frame. Because the CNN structure was a better fit for this specific small dataset, and because we used the Hybrid Loss to let the Student learn from the Ground Truth, the Student surpassed the Teacher.

D. Real World Speed

We tested the speed on a Mac M1 to simulate a mobile chip.

- **Speed:** The Student ran in **11.16 ms** (about 90 frames per second), while the Teacher took 156.9 ms.
- **Size:** The Student was only **47.6 MB**, while the Teacher was a massive 1.6 GB.

VII. DISCUSSION & FUTURE WORK

A. Implications for AR Hardware

Our results show that the Teacher–Student method works great for solving the “Cloud vs. Edge” problem. The latency drop is huge (down to 11ms), which is fast enough for real AR. “Motion-to-photon” latency (the time it takes for a screen to update when you move your head) needs to be very low to prevent motion sickness. Our method fits within that safety buffer.

B. Energy Consumption

While we focused on latency, the implications for energy efficiency are equally critical. The Teacher model (ViT-L) requires substantial GPU wattage, which would drain a standard AR headset battery (approx. 3000-4000 mAh) in minutes. The Student model (MobileNetV3), operating at 47 MB and low FLOPs, falls within the thermal design power (TDP) of modern chips. This suggests our approach not only improves speed but enables “all-day” battery life for continuous AR agents.

C. Limitations

The failure of our simulation attempts (Phase 2 Preliminary) highlights a significant limitation in the current pipeline: Teacher Competence. Distillation is most effective when the Teacher is an expert. In our case, the difficulty of exploration in both high-fidelity sims and the synthetic gridworld meant the Teacher was often suboptimal. Future work must focus on pre-training Teachers on massive offline datasets to surmount the exploration hurdle before distillation begins.

D. Next Steps

This study opens several paths for future research:

- **Hardware Testing:** Future experiments should move beyond MPS simulation to deploy models on dedicated AR hardware to measure thermal efficiency.
- **Federated Learning:** To protect privacy, we could explore Federated Learning. This is where the Student learns on the user’s device and never sends the video data to the cloud at all, only sharing the math updates.

VIII. CONCLUSION

In this project, we built and tested a Teacher–Student pipeline for AR. We showed that we can make AI models **78×** faster and **98%** smaller without losing accuracy. In fact, sometimes the smaller Student model works even better than the big Teacher because it is better suited for the specific data. This suggests that for future AR glasses, “distilling” cloud intelligence onto the device is the most viable path forward.

REFERENCES

- [1] G. Hinton, O. Vinyals, and J. Dean, “Distilling the knowledge in a neural network,” *arXiv preprint arXiv:1503.02531*, 2015.
- [2] J. Schulman et al., “Proximal policy optimization algorithms,” *arXiv preprint arXiv:1707.06347*, 2017.
- [3] A. Howard et al., “Searching for mobilenetv3,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019.
- [4] V. Mnih et al., “Human-level control through deep reinforcement learning,” *Nature*, vol. 518, 2015.
- [5] Y. Li et al., “In the eye of beholder: Joint learning of gaze and actions in first person video,” in *European Conference on Computer Vision (ECCV)*, 2018.
- [6] A. Radford et al., “Learning transferable visual models from natural language supervision,” in *International Conference on Machine Learning (ICML)*, 2021.
- [7] K. Grauman et al., “Ego-Exo4D: Understanding Skilled Human Activity from First- and Third-Person Perspectives,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [8] L. Jiang et al., “EdgeOL: Efficient in-situ Online Learning on Edge Devices,” *arXiv preprint arXiv:2401.16694*, 2024.
- [9] M. Bettini, A. Prorok, and V. Moens, “BenchMARL: Benchmarking Multi-Agent Reinforcement Learning,” in *Proceedings of the 38th Conference on Neural Information Processing Systems (NeurIPS)*, 2024.
- [10] B. A. Kitchenham, T. Dybå, and M. Jørgensen, “Evidence-based software engineering,” *The Journal of Systems and Software*, vol. 72, no. 1, pp. 1–5, 2004.